

Chapter 10

Implementing threads

Chapter 2 used threads; this chapter builds them. The mechanics are the same whether the switching happens in a user-level library (`module 10`) or in a kernel (SWEB): a record per thread, a way to save and restore registers, a stack per thread, a state machine, and wait queues. The hard parts are at the edges — how a thread starts, how it ends, who frees its stack, and what happens when an interrupt arrives in the middle of the library’s own bookkeeping.

10.1 The thread control block and the context

For each thread the implementation keeps a *thread control block* (TCB): id, state, saved registers (the *context*), stack, start routine and argument, return value, detached flag, and links for the queues it may sit in.

A context switch saves the running thread’s registers into its TCB and loads the next thread’s registers from its TCB. Loading the stack pointer switches stacks; loading the instruction pointer continues wherever the next thread stopped. From the thread’s point of view, the call that switched away simply returns — later. `swapcontext(&old, &new)` does exactly this in user space; SWEB’s `arch_contextSwitch` restores a `Thread`’s saved `ArchThreadRegisters`.

10.2 States

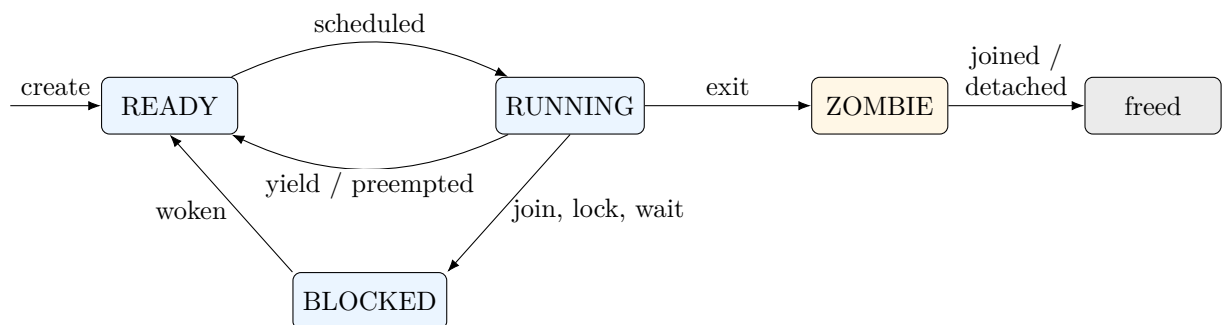


Figure 10.1: Thread states. SWEB’s names: `Running` covers `READY` and `RUNNING`, `Sleeping` is `BLOCKED`, `ToBeDestroyed` is the way out.

A **BLOCKED** thread is in no ready queue; exactly one wait queue (a mutex’s, a condition’s, a joiner slot) knows it, and whoever makes the awaited event happen moves it back to **READY**.

10.3 Birth: a stack and a trampoline

A new thread needs a stack of its own and a first context that “returns” into its code. The first instruction must not be the start routine itself: when `start_routine` returns, it would return to whatever garbage lies on the fresh stack. Instead the thread starts in a small *trampoline*:

```
static void trampoline(void) {
    after_switch();
    pthread_exit(current->start(current->arg));
}
```

/* see "Death" below */
/* return == exit */

Stacks should have a *guard page* below them (mapped without any access rights), so that an overflow faults at once instead of silently overwriting the neighbouring stack.

10.4 Death: exit, zombies, join, detach

- **exit**: store the return value, become a ZOMBIE, wake the joiner if one waits, switch away — forever.
- **join**: if the target is already a zombie, collect and free it; otherwise block until its exit wakes you, then collect and free. Errors: no such thread (ESRCH), joining yourself or closing a cycle of joins (EDEADLK), a detached target or a second joiner (EINVAL).
- **detach**: nobody will join; free the thread as soon as it is a zombie (at once, if it already is).

Nobody frees their own stack. A detached thread that exits cannot unmap its stack — it is still running on it; the next push or return would fault. The freeing has to happen *after* the switch, by somebody else: module 10’s library notes the thread in a variable, and the next thread to run frees it right after the switch (in `after_switch`, which the trampoline also calls). SWEB solves the same problem for kernel stacks with a dedicated thread: `Thread::kill` only marks the thread `ToBeDestroyed`, and the cleanup thread `deletes` it later.

10.5 Blocking primitives

A blocking mutex is an owner plus a FIFO of waiting TCBs:

- lock: free → take it; else enqueue yourself, state BLOCKED, switch away;
- unlock: waiters? either *hand the mutex over* to the first one (FIFO, fair, but every contended unlock forces a switch) or free it and wake a waiter who must compete again (*barging*: faster, unfair — SWEB’s `Mutex` does this).

A condition variable is just a wait queue; `wait` enqueues, unlocks the mutex and blocks *without any switch in between*, then re-locks after being woken. Deadlock detection comes almost for free: if the ready queue is empty but some thread is BLOCKED, nothing can ever wake it.

10.6 Preemption and interrupt safety

Without preemption, a thread runs until it calls into the library; atomicity inside the library is automatic. With a timer (module 10’s bonus uses SIGVTALRM, the kernel uses the timer interrupt), a switch can happen *anywhere* — also in the middle of `q_push` or between “mutex is free” and “I am the owner”. The library’s own data then needs protection: block the signal (user space) or disable interrupts (kernel, one CPU) around every operation on it, exactly as SWEB’s scheduler runs with interrupts off. On several CPUs this is not enough; a spinlock is needed as well.

And code that can be interrupted at any instruction must not be re-entered through non-reentrant functions: a thread preempted inside `malloc` holds `malloc`'s lock; another thread on the same kernel thread calling `malloc` deadlocks. SWEB's version of the rule: no `new/delete` with interrupts off or in interrupt handlers.

10.7 User-level vs. kernel-level, revisited

	User-level (M:1)	Kernel-level (1:1)
switch cost	function call + registers (module 10: $\approx 0.4 \mu\text{s}$, mostly glibc's signal-mask syscall)	kernel entry, scheduler, exit ($\approx 3 \mu\text{s}$ in module 10's measurement)
blocking syscall	blocks <i>all</i> threads	blocks only the caller
multiple cores	no	yes
per thread	TCB, user stack	kernel stack, user stack, two register sets, kernel TCB

In SWEB

After A1 every pthread is a kernel **Thread** (1:1). The pieces map one to one:

- TCB = **Thread**; its `kernel_stack_` is 8 KiB inside the object; `user_registers_` and `kernel_registers_` are the two contexts (user mode, and the kernel side of a system call or interrupt).
- Birth: `ArchThreads::createUserRegisters(regs, entry, user_stack_top, kernel_stack);` `entry` = a libc wrapper that calls `start_routine(arg)` and then `pthread_exit`; the two values reach it in `rdi/rsi`. `setAddressSpace` gives the thread the process's page tables.
- The user stack: a new region in the shared address space, below the first thread's stack page, with a guard gap (chapter 8).
- Death and reaping: `kill` $\rightarrow$ `ToBeDestroyed` $\rightarrow$ cleanup thread. Join data (return value, finished) must survive the **Thread** object — keep it in the process.
- Waiting: `Condition/Mutex` (chapter 5), never a yield loop (chapter 9).

Key points

- A context switch = save registers into one TCB, load them from another; the stack pointer switches the stack.
- Start through a trampoline so that returning means exiting; guard pages catch stack overflows.
- Zombies keep the return value until join; detached threads are freed at exit — by somebody else, after the switch.
- Blocking = a wait queue + state `BLOCKED` + switch; hand-off vs. barging.
- Preemption makes the library's own data shared: disable interrupts / block the signal; no non-reentrant calls.